TECNOLÓGICO NACIONAL DE MÉXICO

TECNOLÓGICO NACIONAL DE MÉXICO CAMPUS CULIACÁN

SISTEMA DE ENRUTAMIENTO PARA TIENDAS DE AUTOSERVICIO (MODULO II)

Topicos de IA

Rodriguez Lopez Said Armando
Santacruz Maldonado Oscar Said

Zuriel Dathan Mora Felix

Ingeniería en Sistemas computacionales
9no Semestre

9:00 - 10:00

16/10/2026

Índice

Introducción 1
Objetivos del Proyecto 2
Objetivo General 2
Objetivos Específicos 2
Justificación 3
Alcance 4
Desarrollo 5
1.1 Descripción del problema 5
1.2 Datos disponibles 5
1.3 Selección y justificación del algoritmo heurístico 5
1.4 Implementación computacional 6
1.5 Resultados obtenidos 7
1.6 Análisis de efectividad 9
Agenda de Trabajo 9
Conclusiones 10
Referencias 11

Introducción

Cuando una empresa tiene varias sucursales, la distribución termina siendo uno de los gastos más fuertes, aunque casi no se le pone tanta atención como a otras áreas. Mover los productos del almacén a cada punto de venta parece fácil, pero si la ruta no está bien pensada el camión hace más kilómetros de los necesarios, se gasta más combustible y el chofer trabaja más horas. Y al final eso también se nota en las sucursales, porque los productos llegan tarde y el cliente es el que lo resiente.

En este trabajo nos tocó el caso de abastecer de abarrotes a 90 sucursales desde 10 centros de distribución (CEDIS). La idea es que cada sucursal se surta del CEDIS que le queda más cerca y luego cada CEDIS decida en qué orden visitar a las suyas para gastar lo menos posible en combustible. Lo complicado es justamente el orden: nada más una zona con 21 sucursales tiene más de 5×10^{19} recorridos posibles, entonces revisarlos todos, ya sea a mano o uno por uno en la computadora, se hace una tarea imposible.

Para esto se usan métodos heurísticos, que no garantizan llegar al óptimo global, pero sí encuentran una solución bastante buena y rápido. El problema de muchos métodos es que se quedan atorados en un óptimo local, que es una ruta que parece la mejor comparada con las que tiene cerca, pero no lo es si se ve todo el problema. Las dos opciones fueron búsqueda tabú y recocido simulado, y decidimos utilizar el recocido simulado porque a veces acepta rutas peores y eso le ayuda a salirse de esos óptimos locales. Lo combinamos con tres formas de cambiar una ruta y con enfriamiento logarítmico, y luego medimos cuánto bajaba el costo de combustible y la distancia.

Objetivos del Proyecto

Objetivo General

Diseñar e implementar un programa basado en un algoritmo heurístico (recocido simulado o búsqueda tabú) que ayude a optimizar las rutas de distribución de productos comestibles desde los centros de distribución hasta las sucursales de una cadena de restaurantes, para que el reparto sea más eficiente y se gaste menos.

Objetivos Específicos

- Analizar cómo funciona la distribución de productos comestibles en la zona de estudio y qué problemas tiene.
- Plantear el problema como uno de optimización combinatoria.
- Escoger el algoritmo heurístico que mejor se adapte al problema y explicar por qué.
- Programar el algoritmo que se haya escogido.
- Probarlo simulando la distribución con datos reales.
- Medir qué tan bien funciona con métricas como el costo y la distancia.
- Documentar todo el proceso del proyecto.

Justificación

Con 100 puntos en la red, el número de rutas posibles crece tan rápido que ni un método manual ni un método exacto pueden revisarlas en un tiempo razonable. Los métodos heurísticos resuelven este inconveniente cambiando la búsqueda de la ruta perfecta por la búsqueda de una ruta muy buena en segundos. Para la empresa esto significa poder recalcular el plan de reparto cada vez que cambian las condiciones, sin depender de la experiencia de un planeador. El recocido simulado es especialmente útil porque, al aceptar de vez en cuando rutas peores, evita quedarse con la primera solución aceptable que encuentra y sigue explorando hasta dar con recorridos más baratos. En la práctica, esto se traduce en menos kilómetros, menos combustible y entregas más ordenadas, y le da a quien toma las decisiones una herramienta medible para comparar alternativas.

Alcance

El proyecto se limita a optimizar las rutas de distribución para 100 nodos (10 centros de distribución y 90 tiendas), usando recocido simulado para bajar la distancia recorrida y el costo de combustible. El programa carga los datos, asigna cada tienda a su CEDIS, revisa que la demanda de las tiendas de cada CEDIS quepa en su capacidad, genera la solución inicial y las vecinas, y compara las rutas por distancia total, costo y porcentaje de mejora. Hay cosas que no se tomaron en cuenta, como el tráfico, el clima u otros imprevistos, la capacidad de carga de los camiones o usar más de un camión por CEDIS. Lo que se busca es tener rutas optimizadas que sirvan como apoyo para tomar decisiones, con sus tablas, gráficas e impresiones de la ejecución.

Desarrollo

1.1 Descripción del problema

El problema es optimizar las rutas de distribución en Culiacán, Sinaloa, con 100 nodos: 10 son centros de distribución (CEDIS) y 90 son tiendas. Cada ruta sale de un CEDIS, pasa una sola vez por cada tienda que le toca y regresa al mismo CEDIS. Como cada tienda se surte desde el CEDIS más cercano, en realidad el problema se puede partir en diez problemas independientes, uno por CEDIS.

Lo primero que se hace es asignar cada tienda al CEDIS que tiene más cerca, o sea, al que está a menor distancia. Después, para cada CEDIS se busca en qué orden conviene visitar sus tiendas para que el costo total de la ruta sea el menor posible. La función objetivo es la suma de los costos de combustible entre cada par de nodos que van seguidos en el recorrido, por lo que si se reduce la distancia también se reduce el costo. Además, cada tienda tiene un nivel (A, B o C) que equivale a cierta demanda, y se revisa que la demanda total de las tiendas de cada CEDIS no pase de su capacidad de almacenamiento.

1.2 Datos disponibles

- **datos_distribucion_tiendas.xlsx:** contiene, para cada uno de los 100 nodos, su tipo (centro de distribución o tienda), su nombre, sus coordenadas geográficas, su capacidad de venta y de almacenamiento y, en el caso de las tiendas, su nivel (A, B o C). Hay 33 tiendas de nivel A, 25 de nivel B y 32 de nivel C.
- **matriz_distancias.xlsx:** matriz de 100 × 100 con la distancia en kilómetros entre todos los nodos. Estas son las distancias que usa el algoritmo, no las que saldrían de las coordenadas.
- **matriz_costos_combustible.xlsx:** matriz de 100 × 100 con el costo de combustible entre cada par de nodos. El costo es proporcional a la distancia, a razón de 0.15 pesos por kilómetro.

Para revisar la demanda supusimos que una tienda de nivel A pide 1200 unidades, una de nivel B 1000 y una de nivel C 400. La capacidad de almacenamiento de cada CEDIS viene en el mismo archivo y va desde 11 639 hasta 49 453 unidades.

1.3 Selección y justificación del algoritmo heurístico

Se compararon los dos algoritmos propuestos para el proyecto. La búsqueda tabú revisa en cada paso muchos vecinos de la solución actual, se mueve al mejor de ellos y guarda una lista de movimientos prohibidos para no regresar a soluciones recientes. Es muy eficaz, pero cada iteración es costosa y su desempeño depende del tamaño de la lista tabú y de los criterios de aspiración. El recocido simulado se inspira en el enfriamiento controlado de los metales: en cada paso genera un solo vecino y, si es peor, todavía puede aceptarlo con una probabilidad que disminuye conforme baja la temperatura.

Se eligió el recocido simulado principalmente porque su mecanismo para escapar de óptimos locales se controla fácilmente con la temperatura, además que no necesita memoria adicional. Además, tiene una base teórica sólida: con un enfriamiento logarítmico suficientemente lento converge al óptimo global, y se ha aplicado con buenos resultados al problema del agente viajero desde sus primeros trabajos. Su implementación es flexible, ya que se ajusta con parámetros como

la temperatura inicial, el número de pruebas por temperatura y los criterios de parada, lo que lo hace adecuado para zonas de entre 2 y 21 tiendas como las de este proyecto.

1.4 Implementación computacional

El programa se hizo en Python y se dividio en módulos, uno para cargar los datos, otro para asignar las tiendas a cada CEDIS, otro para la demanda, el de la optimización, el de análisis de resultados y el principal, que tiene el menú.

En la carga de datos se leen los tres archivos de Excel, se revisa que las matrices tengan el tamaño correcto, se pone en cero la diagonal y se separan los nodos entre CEDIS y tiendas. Con esto ya se puede hacer la asignación de tiendas, en la que se compara la distancia de cada tienda con los diez CEDIS y se le asigna el que le queda más cerca, de esta forma el problema queda dividido en diez problemas más pequeños. Luego, en la revisión de la demanda se suma la demanda de las tiendas que le tocaron a cada CEDIS según su nivel, se compara con la capacidad que tiene y a cada CEDIS se le marca como OK o SOBRECARGADO.

La optimización es donde está el recocido simulado. Para cada CEDIS se arranca con una ruta al azar que sale del CEDIS y regresa a él, y el costo es la suma del combustible entre cada par de nodos seguidos. Los vecinos se hacen con uno de tres movimientos que se escoge al azar (el CEDIS del inicio y del final no se mueve): swap, que cambia dos tiendas de lugar, inserción, que saca una tienda y la mete en otra posición, y 2-opt, que voltea un pedazo de la ruta, esto sirve para quitar cruces. Si el vecino sale más barato se acepta y si sale más caro se acepta con probabilidad $\exp(-\Delta/T)$ (la de Boltzmann), donde Δ es cuánto más caro salió y T la temperatura.

La temperatura se va bajando con $T = T_0 \cdot \ln 2 \,/\, \ln(k + 2)$, donde k es el nivel. En cada nivel se prueban 50 vecinos y se para cuando se llega a 1000 niveles, cuando van 150 niveles seguidos sin mejorar o cuando la temperatura llega al mínimo. También se puede repetir el recocido varias veces en el mismo CEDIS (arranques) y quedarse con la mejor ruta. En cada nivel se guarda la temperatura, el costo actual y el mejor costo, porque eso es lo que se usa después para las gráficas.

El módulo de análisis toma eso, arma la tabla por CEDIS, la guarda en Excel y hace las gráficas de temperatura y costo. Desde el menú principal se puede correr todo, cambiar los parámetros (vienen ya con los recomendados), ver resultados o graficar un CEDIS.

1.5 Resultados obtenidos

Para probar el algoritmo corrimos el programa con los parámetros recomendados (temperatura inicial de 0.5, 50 vecinos por nivel, máximo 1000 niveles y 150 niveles sin mejora para detenerse) y comparamos la solución inicial, que es aleatoria, contra la que entrega el recocido simulado. En la Tabla 1 están los resultados generales.

Métrica	Solución inicial	Solución final	Diferencia
Costo de ruta (MXN)	64.16	30.42	−33.74
Distancia total (km)	427.75	202.86	−224.89
Mejora en costo (%)	–	–	52.59%
Mejora en distancia (%)	–	–	52.57%
Tiendas atendidas	90	90	–
Demanda total (unidades)	77 400	77 400	–
CEDIS que mejoraron	–	9 de 10	–
Tiempo de ejecución	–	≈ 1 s	–

Tabla 1: Resultados generales de la ejecución

Lo que más llama la atención es que el costo y la distancia bajaron a menos de la mitad: 33.74 pesos y casi 225 kilómetros menos. Puede parecer raro que los dos porcentajes salgan casi iguales (52.59% y 52.57%), pero tiene sentido, porque el costo de combustible es proporcional a la distancia (0.15 pesos por kilómetro), así que si la ruta se acorta, el costo baja en la misma proporción. Además se siguieron atendiendo las 90 tiendas, ningún CEDIS quedó sobrecargado y todo tardó un segundo.

Resultados por CEDIS

El programa también imprime una tabla con los resultados de cada CEDIS (Figura 1), donde viene cuántas tiendas tiene, la distancia y el costo antes y después, la mejora, la demanda, la capacidad, qué porcentaje de esa capacidad se usa y si quedó OK o SOBRECARGADO.

```
  Opcion: 3
CEDIS  Tiendas  Dist_Inicial  Dist_Final  Costo_Inicial  Costo_Final  Mejora  Mejora_%  Demanda_Zona  Capacidad  Uso_%  Estado
    0        5        17.26       13.56           2.59         2.03    0.55     21.42          3800    11639.0  32.65      OK
    1        6        29.30       16.31           4.40         2.45    1.95     44.35          4600    26049.0  17.66      OK
    2        4        13.87       12.15           2.08         1.82    0.26     12.41          4600    19144.0  24.03      OK
    3       21       115.68       33.41          17.35         5.01   12.34     71.12         19200    45741.0  41.98      OK
    4       11        53.08       23.37           7.96         3.51    4.46     55.98          8000    48698.0  16.43      OK
    5        8        23.56       15.22           3.53         2.28    1.25     35.41          6800    12082.0  56.28      OK
    6       17        86.00       31.93          12.90         4.79    8.11     62.87         14000    16140.0  86.74      OK
    7        2        11.29       11.29           1.69         1.69    0.00      0.00          1600    25247.0   6.34      OK
    8       12        57.45       30.88           8.62         4.63    3.99     46.24         11000    49453.0  22.24      OK
    9        4        20.26       14.74           3.04         2.21    0.83     27.26          3800    46781.0   8.12      OK
TOTAL       90       427.75      202.86          64.16        30.42   33.74     52.59         77400         -      -       -
  Excel: salida/Resultados.xlsx
```

Figura 1: Métricas finales del algoritmo

Ahí se nota que los CEDIS con más tiendas son los que más ahorraron. El CEDIS 3, que tiene 21, pasó de 17.35 a 5.01 pesos y el CEDIS 6, con 17, de 12.90 a 4.79. Los que tienen pocas tiendas mejoraron menos, por ejemplo el CEDIS 2 (4 tiendas, 12.41%), y el CEDIS 7 se quedó igual porque con 2 tiendas no hay nada que reordenar. De capacidad no hubo problema: el más cargado es el CEDIS 6 con 86.74% y el más libre el CEDIS 7 con 6.34%.

Análisis específico para el CEDIS 8

Para ver más a detalle cómo se comporta el algoritmo escogimos el CEDIS 8, que atiende 12 tiendas. Está más o menos a la mitad en cuanto a tamaño y su mejora (46.24%) no se aleja mucho de la general. Aquí el costo bajó de 8.62 a 4.63 pesos y la distancia de 57.45 a 30.88 kilómetros.

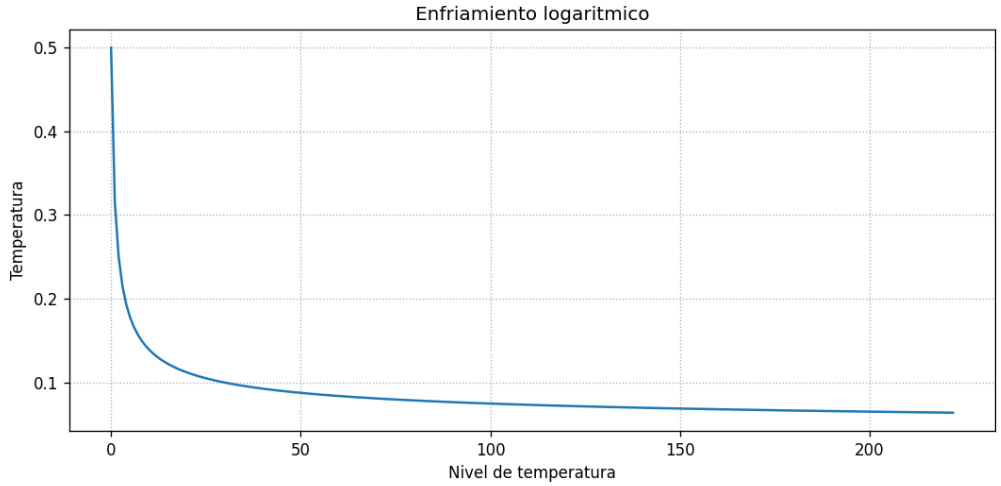


Figura 2: Evolución de la temperatura en el algoritmo para el CEDIS 8

La Figura 2 muestra cómo fue bajando la temperatura en cada nivel. La corrida empieza en 0.5 y en los primeros cuatro niveles ya baja a menos de 0.2, o sea que pierde más de la mitad de su valor muy rápido. Después la caída se hace mucho más lenta: cerca del nivel 30 la temperatura anda por 0.1 y de ahí hasta el final solo baja a poco más de 0.06. Esto pasa por la forma de la fórmula, ya que el logaritmo que está en el denominador crece cada vez más despacio. La gráfica termina un poco después del nivel 220 y no en el 1000, porque el algoritmo se detuvo al cumplir 150 niveles seguidos sin encontrar una ruta mejor.

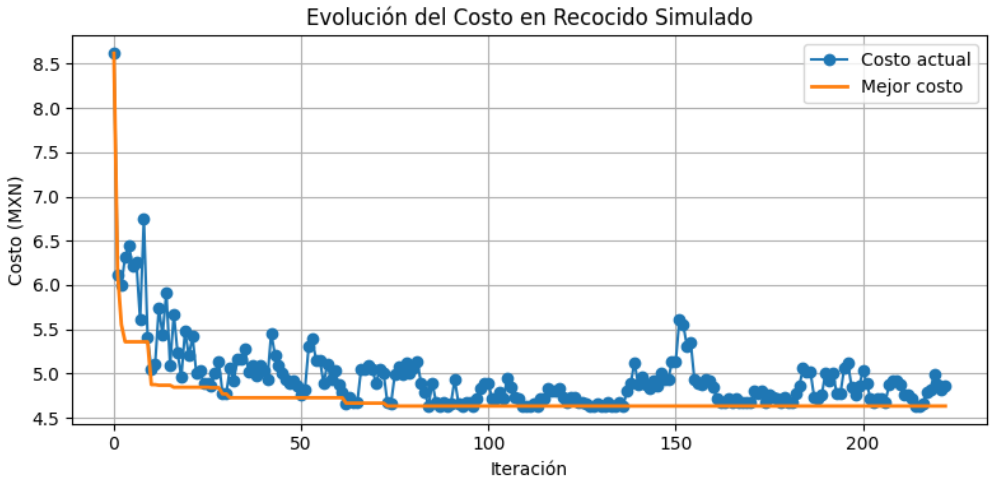


Figura 3: Evolución del costo durante el recocido simulado en el CEDIS 8

En la Figura 3 se compara el costo de la ruta actual (línea azul) con el mejor costo encontrado hasta ese momento (línea naranja). Las dos empiezan en 8.62, que es el costo de la ruta aleatoria. En las primeras diez iteraciones el mejor costo baja de golpe, primero a cerca de 5.35 y luego a

4.87, mientras que el costo actual brinca bastante y hasta llega a subir a cerca de 6.75, porque con la temperatura alta el algoritmo acepta muchas rutas peores. Entre las iteraciones 10 y 70 el mejor costo sigue bajando en escalones pequeños, a cerca de 4.73 y después a 4.63, que ya es el valor final. Desde ahí la línea naranja se queda plana, pero la azul sigue moviéndose más o menos entre 4.6 y 5.1, y alrededor de la iteración 150 tiene otro salto hasta casi 5.6, lo que muestra que el algoritmo seguía probando otras rutas aunque ya no encontrara una mejor. Como después de llegar a 4.63 pasaron 150 niveles sin mejora, la ejecución terminó cerca de la iteración 220.

1.6 Análisis de efectividad

Con los resultados se puede decir que el recocido simulado sí funcionó. Nueve de los diez CEDIS mejoraron y los que más bajaron fueron los que tienen más tiendas, el CEDIS 3 con 71.12% y el CEDIS 6 con 62.87%, lo cual tiene lógica porque entre más tiendas hay más formas de ordenar la ruta y una ruta al azar sale peor. El CEDIS 7 no cambió, pero tiene 2 tiendas y no hay otro orden. Separar por CEDIS también ayudó, así el algoritmo nunca trabaja con las 90 tiendas juntas.

En la Figura 3 la línea azul nunca se queda quieta, sigue agarrando rutas más caras hasta el final porque la temperatura nunca llegó al mínimo. Pero la naranja ya estaba en 4.63 antes de la iteración 70, así que el resto de la corrida ya no encontró nada mejor.

Aun así, los resultados dependen bastante de cómo se configuren los parámetros. En otras pruebas que se hicieron, con una temperatura inicial de 50 el algoritmo aceptaba demasiadas rutas peores y terminó con un costo total de 40.07, mientras que con los valores recomendados casi siempre se llega a un costo cercano a 30.42.

Agenda de Trabajo

En la Tabla 2 se ve el cronograma de las actividades que hicimos durante el proyecto, entre el 10 y el 12 de octubre de 2026.

Fecha	Actividad
10 de octubre	Análisis del problema, revisión de los datos y diseño de la estructura modular del sistema.
11 de octubre	Carga de datos, asignación de las tiendas a cada CEDIS y verificación de la demanda contra la capacidad de cada CEDIS.
11 de octubre	Implementación del recocido simulado con vecindario mixto (swap, inserción y 2-opt), enfriamiento logarítmico y arranques.
12 de octubre	Desarrollo del análisis de resultados, las gráficas y el menú principal.
12 de octubre	Calibración de parámetros, ejecución del programa y elaboración del reporte final.

Tabla 2: Cronograma de actividades del proyecto

Conclusiones

Con este proyecto se pudo aplicar el recocido simulado a un problema real de distribución con 10 centros de distribución y 90 tiendas. Usando los parámetros recomendados, el costo total de las rutas bajó de 64.16 a 30.42 pesos y la distancia recorrida de 427.75 a 202.86 kilómetros, lo que demuestra que los métodos heurísticos son una buena alternativa cuando hay tantas combinaciones que no se pueden usar métodos exactos. Dividir el problema por CEDIS, asignando cada tienda al más cercano, permitió trabajar con grupos más pequeños, y la revisión de la demanda sirvió para saber rápido si cada CEDIS tenía capacidad suficiente para sus tiendas.

Durante el desarrollo también se pudo ver cómo se comporta el algoritmo: al principio explora mucho y acepta rutas peores, y conforme baja la temperatura se enfoca en mejorar las mejores rutas que encontró. Algo que se aprendió es que la temperatura inicial es el parámetro que más afecta el resultado, que usar varios tipos de movimiento es mejor que depender de uno solo, y que tener el código separado en módulos permite cambiar partes del algoritmo sin descomponer lo demás. En general, el proyecto ayudó a reforzar lo visto sobre optimización combinatoria y algoritmos heurísticos, y también a darse cuenta de lo importante que es revisar bien los datos de entrada.

Referencias

- Baeldung. (2024, 25 de enero). The traveling salesman problem in Java. Baeldung. https://www.baeldung.com/java-simulated-annealing-for-traveling-salesman

- Beukers, F. (s.f.). Traveling salesman problem (TSP) using simulated annealing. Universiteit Utrecht. Recuperado el 15 de octubre de 2026, de https://webspace.science.uu.nl/~beuke106/anneal/anneal.html

- GeeksforGeeks. (2024, 8 de abril). Simulated annealing. https://www.geeksforgeeks.org/simulated-annealing/

- McCaffrey, J. (2021, 1 de diciembre). Simulated annealing optimization using C# or Python. Visual Studio Magazine. https://visualstudiomagazine.com/articles/2021/12/01/traveling-salesman.aspx

- Melián Batista, B., & Glover, F. (s.f.). Introducción a la búsqueda tabú. Universidad de La Laguna; University of Colorado at Boulder. Recuperado el 15 de octubre de 2026, de https://leeds-faculty.colorado.edu/glover/fred%20pubs/329%20-%20Introduccion%20a%20la%20Busqueda%20Tabu%20TS_Spanish%20w%20Belen%2811-9-06%29.pdf